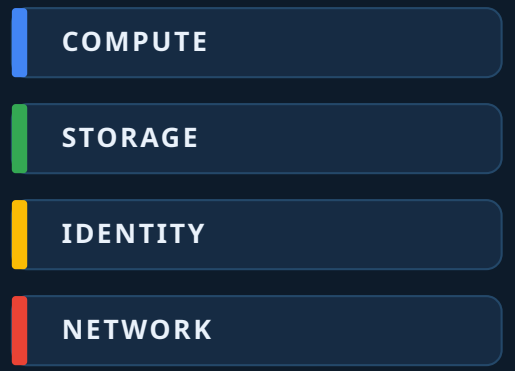
DAY 05 LAB

GOOGLE CLOUD PLATFORM

Theory & Support Scenario Guide

A student-friendly foundation for the Terraform lab: understand the platform, compare AWS / Azure / GCP, troubleshoot four realistic incidents, and capture evidence safely.



START HERE

How to use this deck

Use the theory first, then follow the lab runbook for exact commands and environment-specific values.

01 Learn the platform

- Projects, regions, zones and core services
- Cloud console, gcloud and Terraform
- IAM principals, roles and policies

02 Work the incidents

- Cloud SQL versus VM OS access
- Project / zone discovery
- Least privilege and Deny precedence

03 Prove and clean up

- Capture CLI output and Console screenshots
- Record cause, remediation and result
- Review destroy plan before cleanup

Source boundaries Lab-specific resources, variable names and scenario intent follow the supplied “GCP Day 5 Student Lab Reference”. Platform explanations and comparison notes are expanded using official product documentation. KodeKloud is included as optional supplementary learning.

SAFETY FIRST

Use a disposable training project only. Keep Terraform state private. Never test IAM with a production identity.

Google Cloud overview + supplied student lab guide (pages 1–3)

What is Google Cloud Platform?

A portfolio of infrastructure, platform, data, security and AI services delivered through a common control plane.

Compute

- Compute Engine VMs
- Google Kubernetes Engine
- Cloud Run

Storage & databases

- Cloud Storage buckets/objects
- Persistent Disk / Filestore
- Cloud SQL

Networking

- VPC + subnets
- Firewall rules / routes
- Load balancing / VPN

Identity & operations

- Cloud IAM
- Cloud Monitoring / Logging
- Cloud Billing + budgets

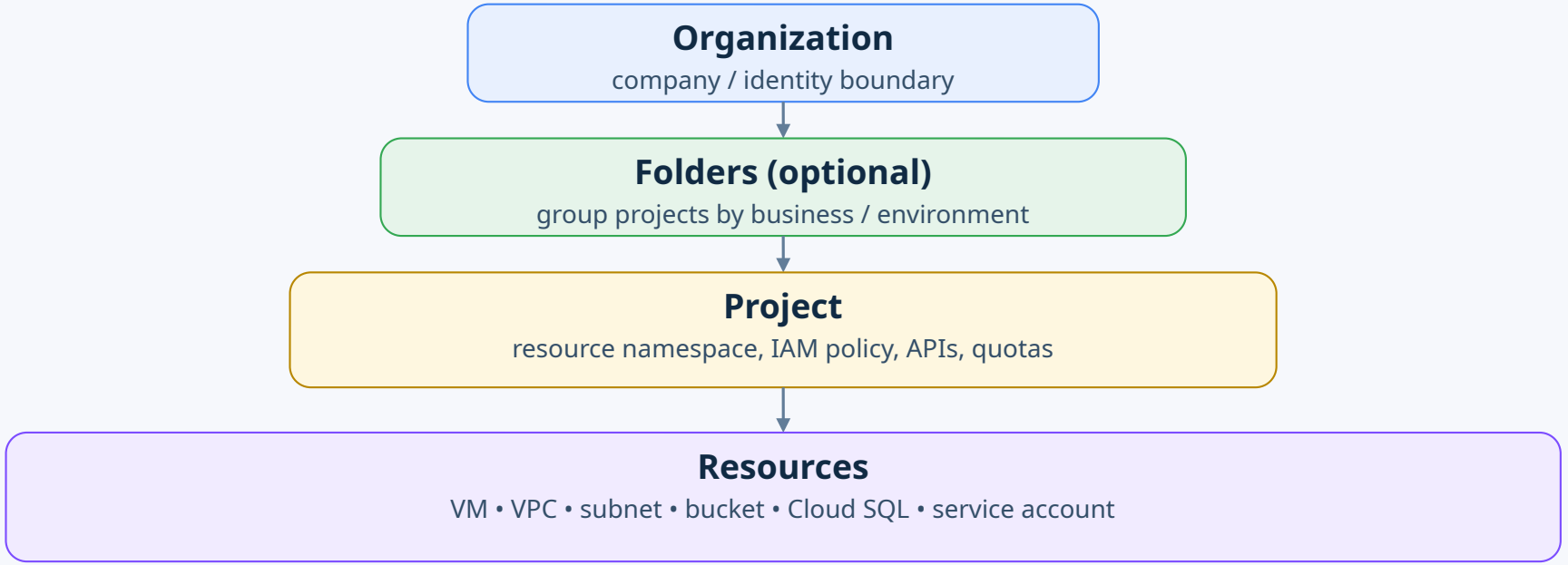
One platform, several ways to operate Google Cloud Console • gcloud CLI • service APIs • Terraform / Infrastructure as Code

This Day 5 lab focuses on Compute Engine, VPC, Cloud Storage, Cloud SQL, IAM and Terraform.

Official: Google Cloud overview and Terraform documentation

Resource hierarchy: where your resource lives

Support triage starts by locating the resource within the organization and the correct project.



Project is not the same as billing account

A project is linked to a billing account; one billing account can pay for multiple projects.

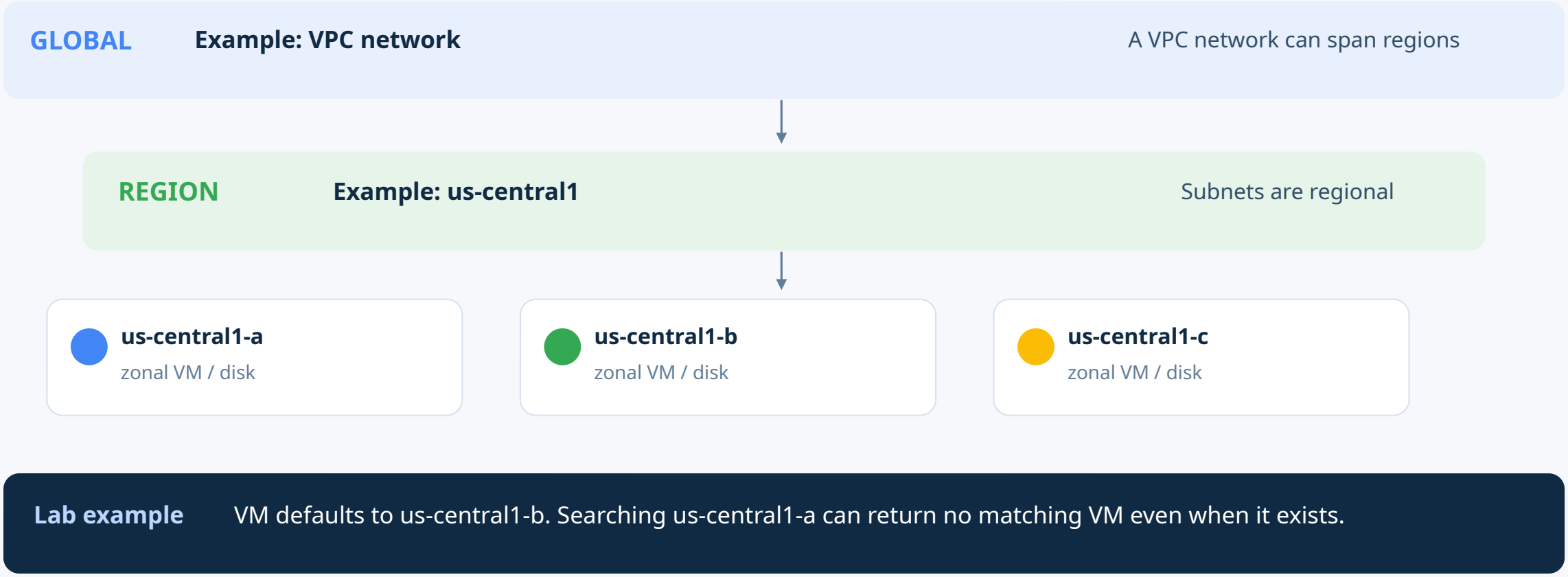
Resource scope affects search

The same VM name may be reused in different projects; a zonal VM must be queried in its actual zone.

Official: Google Cloud overview and Cloud Storage hierarchy

Global, regional and zonal resources

“Location” is not a decorative field: it determines where certain resources exist and how you find them.



Lab example VM defaults to us-central1-b. Searching us-central1-a can return no matching VM even when it exists.

Official: Google Cloud overview; VPC is global with regional subnets

CLOUD COMPARISON

AWS vs Azure vs Google Cloud: quick map

Compare the operating concepts; exact permissions and policy mechanics differ across providers.

Concept	AWS	Microsoft Azure	Google Cloud
Primary container	Account	Subscription	Project
Grouping / governance	Organizations / OUs	Management groups / resource groups	Organization / folders / projects
Geographic hierarchy	Region → Availability Zone	Region → Availability Zone	Region → Zone
Compute VM	Amazon EC2	Azure Virtual Machines	Compute Engine
Object storage	Amazon S3	Azure Blob Storage	Cloud Storage
Managed relational DB	Amazon RDS	Azure SQL Database / Azure Database services	Cloud SQL
Identity & access	IAM users / roles / policies	Microsoft Entra ID + Azure RBAC	Cloud IAM principals / roles / bindings
CLI / IaC	AWS CLI / CloudFormation or Terraform	Azure CLI / ARM or Bicep / Terraform	gcloud CLI / Terraform
Deny / org guardrail	SCP sets max permissions; does not grant access	Deny assignment (generally Azure-managed)	IAM Deny policy blocks matching permissions

GCP billing nuance: projects link to billing accounts; one billing account can pay for multiple projects. Policy note: AWS SCPs set a permissions ceiling (not a grant); Azure deny assignments are generally Azure-managed; GCP IAM Deny blocks matching permissions.

Official references: AWS account model; Azure resource organization; Google Cloud project hierarchy

CLOUD COMPARISON

Service mapping across the three clouds

Use this as a navigation cheat sheet; similar names do not imply identical features or pricing.

Virtual machine	EC2	Azure VM	Compute Engine
Private network	VPC	VNet	VPC network
Subnet	Subnet	Subnet	Subnet (regional)
Object storage	S3 bucket / object	Blob container / blob	Cloud Storage bucket / object
Managed relational DB	RDS	Azure SQL / Azure Database	Cloud SQL
Identity and access	AWS IAM	Entra ID + Azure RBAC	Cloud IAM
Activity / observability	CloudTrail, CloudWatch	Azure Monitor, Activity Log	Cloud Logging, Cloud Monitoring
Infrastructure as code	CloudFormation / Terraform	Bicep / ARM / Terraform	Terraform provider for Google Cloud

Lab focus: Compute Engine + VPC + Cloud Storage + Cloud SQL + Cloud IAM + Terraform

Official: Compute Engine, Cloud Storage, Cloud SQL, IAM, VPC and Terraform docs

Shared responsibility: VM vs managed database

The service model changes which operating-system and platform tasks are yours to perform.

COMPUTE ENGINE VM | IaaS

Customer / support team typically manages

- Guest OS configuration and patching
- Installed agents, packages and runtime
- OS users, SSH access and hardening
- Workload and application processes
- VM sizing, disk and backup design

CLOUD SQL | MANAGED DATABASE

Google manages the underlying service platform

- Underlying host OS and platform operations
- Service maintenance and updates
- Managed backups / HA options as configured
- Platform monitoring and infrastructure
- Customer still manages data, users, access and workload design

Support lesson: “I cannot SSH to the database host” is not necessarily an incident; it may be the expected managed-service boundary.

Official: Compute Engine overview; Cloud SQL overview

CORE OPERATIONS

Core GCP services used by the lab

Know each service’s purpose before you troubleshoot its symptoms.

VM

Compute Engine

Virtual machines, machine types, disks, zones and instance-level operations.

OBJ

Cloud Storage

Buckets contain objects. IAM can grant access to read object data.

SQL

Cloud SQL

Managed relational databases with provider-managed infrastructure operations.

IAM

Cloud IAM

Principals receive roles containing permissions on resources through policies/bindings.

VPC

VPC network

Virtual network, regional subnets, routes and firewall rules.

IaC

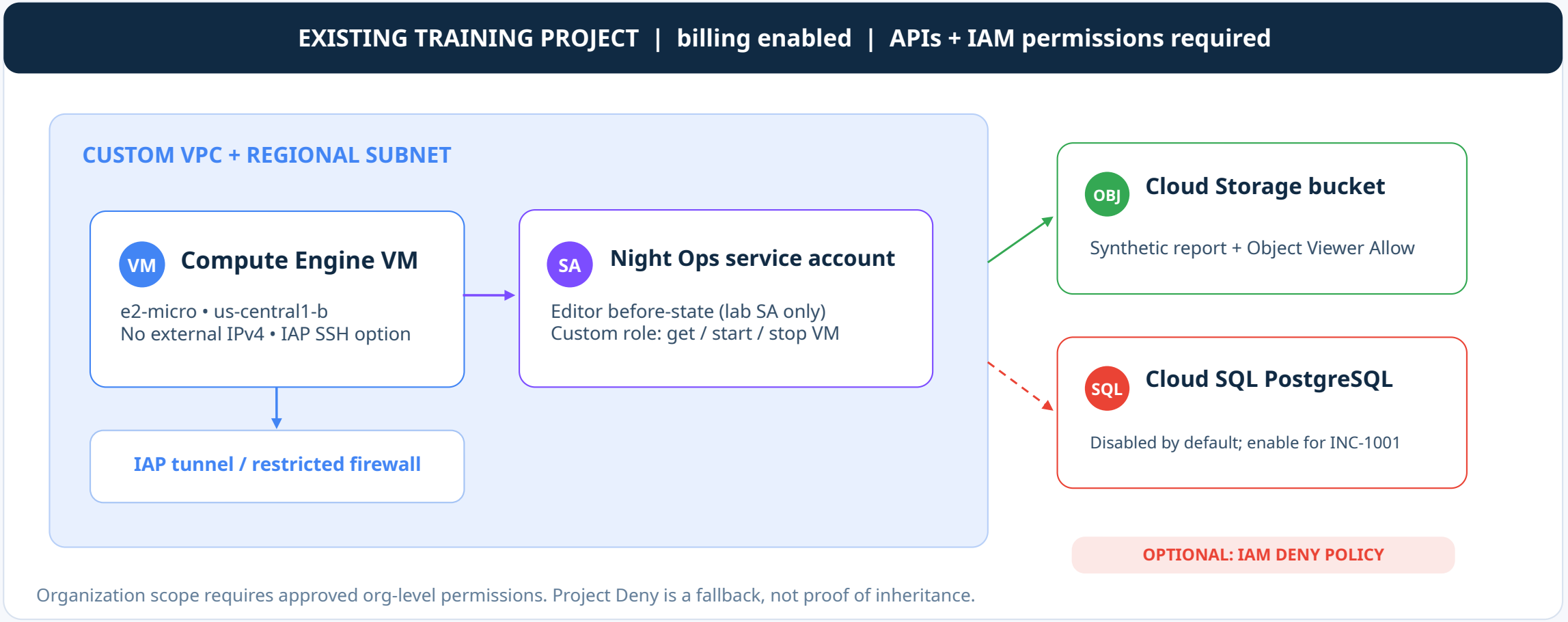
Terraform

Declarative resource provisioning via provider, configuration, plan and state.

Official: service overviews linked in the final reference slides

Day 5 lab: baseline architecture at a glance

Terraform provisions the baseline in an existing billed project; optional scenarios are enabled only when needed.



Organization scope requires approved org-level permissions. Project Deny is a fallback, not proof of inheritance.

Based on the supplied GCP Day 5 Student Lab Reference, pages 2 and 6–12

Three operating paths: Console, gcloud and Terraform

Use each interface for the work it does best; learn to confirm the same fact in more than one place.

Cloud Console

Visual inspection

- Resource details and status
- IAM bindings / permissions
- Activity logs and screenshots

Best for: visual proof

gcloud CLI

Fast and repeatable queries

- Check active project/account
- Describe VM, SQL, IAM context
- Capture terminal output

Best for: triage evidence

Terraform

Desired state and changes

- Define resources in .tf files
- Review plan before apply
- Track owned infrastructure in state

Best for: controlled changes

Best practice: CLI observation + Console screenshot + reviewed Terraform plan = stronger operational evidence.

Official: Google Cloud overview and Terraform basics

Terraform workflow: preview before changing cloud

A plan is a safety checkpoint, not a formality. The lab uses Terraform to create a consistent baseline.

1

init

Download providers and initialize working directory

2

fmt / validate

Format files and check configuration syntax

3

plan

Preview proposed creates, updates and deletes

4

show + review

Confirm project, names, IAM and optional features

5

apply

Execute only after the plan is understood

TYPICAL SEQUENCE

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
```

State file is sensitive

It may contain generated credentials, identifiers and IAM configuration. Keep the lab folder private; do not publish state.

Official: Terraform on Google Cloud; installation and basic commands

Baseline resources and their purpose

The initial apply should be intentionally small; scenario-only resources remain disabled until needed.

Custom VPC + subnet	Isolated network; do not assume the default VPC exists.	**Created**
Compute Engine VM	e2-micro in us-central1-b; no external IPv4; IAP SSH option.	**Created**
Night Ops service account	Dedicated identity for permission tests; no JSON key.	**Created**
Editor before-state	Deliberately broad role assigned only to the lab service account.	**Created for demo**
Custom role + VM binding	get / start / stop; binding is initially disabled.	**Role created**
Bucket + synthetic object	Synthetic report file with Object Viewer Allow.	**Created**
Cloud SQL PostgreSQL	Enable only for INC-1001; billable.	**Disabled**
IAM Deny / second-project VM	Require extra permissions or second billed project.	**Disabled**

Before Apply: verify project_id, lab_user_email, zone, resource names and that optional billable features are disabled.

Supplied GCP Day 5 Student Lab Reference, pages 2, 5–6

SUPPORT METHOD

Cloud support triage: check context before action

A missing resource, access denial or unexpected output often starts with the wrong context—not a broken resource.

1 Identity

Which user / service account is making the call?

2 Project

Is the active project the expected project ID?

3 Location

Is the resource zonal or regional? Is the zone correct?

4 Filters

Do name, status, labels and console filters match?

5 Permissions

Which principal, permission and policy apply?

6 Evidence

Capture before / after output, then remediate safely.

Never create a duplicate VM or widen access until the original asset, intended action and minimum required permission are established.

Concepts aligned to Compute Engine listing, IAM overview and Policy Troubleshooter docs

SCENARIO 01

GCP-INC-1001 | Cloud SQL vs Compute Engine

Scenario objective: explain the service boundary using actual metadata and a real guest-OS check.

COMPUTE ENGINE VM

- You control the guest operating system.
- You can SSH (if access is configured).
- You can inspect hostname and OS release.
- OS patching and installed software are your responsibility.

CLOUD SQL POSTGRESQL

- Google manages the underlying host OS.
- You inspect database instance metadata and settings.
- No customer SSH/root route to the database host.
- You still manage database users, data and access choices.

Evidence to capture Cloud SQL describe output + VM hostname + /etc/os-release + Console details + a clear responsibility conclusion.

Official: Cloud SQL overview; Compute Engine overview

SCENARIO 01

INC-1001 | Scenario flow and command examples

Keep Cloud SQL off until this scenario; enable it, capture proof, then turn it off again.

A ENABLE + INSPECT CLOUD SQL

CLOUD SHELL / BASH

```
# In terraform.tfvars
enable_cloud_sql = true

terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan

PROJECT_ID="$(terraform output -raw project_id)"
SQL_NAME="$(terraform output -raw cloud_sql_instance_name)"
gcloud sql instances describe "$SQL_NAME" \
  --project="$PROJECT_ID" \
  --format="yaml(name,region,databaseVersion,instanceType,settings.tier)"
```

B PROVE GUEST-OS ACCESS ON THE VM

CLOUD SHELL / BASH

```
VM_NAME="$(terraform output -raw vm_name)"
VM_ZONE="$(terraform output -raw vm_zone)"

gcloud compute ssh "$VM_NAME" \
  --project="$PROJECT_ID" \
  --zone="$VM_ZONE" --tunnel-through-iap

# Inside the VM
hostname
cat /etc/os-release
exit
```

After evidence: set enable_cloud_sql = false, review the plan and apply the change. Check the selected tier, quota and cost before enabling the database.

Command structure follows supplied student lab guide, page 7; validate current Cloud SQL tier availability

SCENARIO 02

GCP-INC-1002 | Project and zone discovery

Scenario objective: locate the VM without creating a duplicate. An empty list is not enough evidence that a resource is gone.

Wrong context

Wrong project / zone / filters

Result: lab VM is missing from this query.

Validate context

Check account + project + zone

Confirm outputs and actual resource identifiers.

Correct discovery

Query actual project and zone

Result: VM appears with current status / details.

CLOUD SHELL / BASH

```
gcloud config get-value project
gcloud config get-value compute/zone

# Intentionally wrong zone (lab VM defaults to us-central1-b)
gcloud compute instances list --project="$PROJECT_ID" \
  --zones="us-central1-a" --format="table(name,status,zone)"

# Query correct zone
gcloud compute instances list --project="$PROJECT_ID" \
  --zones="$VM_ZONE" --format="table(name,status,zone)"
```

Console proof

- Project picker shows intended project
- Zone filter shows actual zone
- VM details record status + IPs

Official: Compute Engine list instances documentation

SCENARIO 03

IAM essentials: identity + role + resource

IAM answers: who is acting, which permissions are available, and on what resource.

1 **PRINCIPAL**

Who?

User • group • service account

→

2 **ROLE**

What can they do?

A set of permissions

→

3 **RESOURCE**

Where?

Project • VM • bucket

Role types

- Basic: broad roles such as Owner / Editor / Viewer
- Predefined: Google-managed, service-focused permissions
- Custom: selected permissions for a narrow requirement

Least privilege

Grant only the permissions required, to the right principal, at the narrowest practical scope. Prefer predefined roles when appropriate; use custom roles when necessary.

Lab custom role: compute.instances.get • compute.instances.start • compute.instances.stop

Official: IAM overview and roles overview

SCENARIO 03

GCP-INC-1003 | From broad Editor to narrow VM access

Capture the before-state first. Then remove Editor from the lab identity and bind a custom role directly to the named VM.

BEFORE: BROAD PROJECT ACCESS

CAPTURE BASELINE EVIDENCE

```
# Confirm Editor binding for the lab service account
gcloud projects get-iam-policy "$PROJECT_ID" \
  --flatten="bindings[].members" \
  --filter="bindings.members:serviceAccount:${SA_EMAIL}" \
  --format="table(bindings.role,bindings.members)"

Expected: roles/editor at project scope
```

AFTER: CUSTOM ROLE ON ONE VM

REMEDIATE + TEST ALLOWED ACTION

```
# In terraform.tfvars set together:
seed_editor_binding = false
enable_custom_role_binding = true

terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan

# Safe positive test
gcloud --impersonate-service-account="$SA_EMAIL" \
  compute instances describe "$VM_NAME" \
  --project="$PROJECT_ID" --zone="$VM_ZONE"
```

Expected after-state

Editor no longer appears at project scope for this lab SA; custom role is attached to the one target VM.

Safe negative test

Project-wide instance list should fail because compute.instances.list is not in the custom role. Do not use a destructive test.

Do this only with the disposable training identity. Never use a production service account or grant Editor to a person for this demo.

Based on supplied lab guide, pages 9–10; official custom IAM role guidance

SCENARIO 04

IAM Deny: why an Allow may not be enough

The lab uses the same object-read command before and after a Deny policy is applied.

1 ALLOW EXISTS

roles/storage.objectViewer

The test service account can read the synthetic report object.



DENY

Applicable rule blocks object read

2 REQUEST DENIED

Same principal
Same object
Same read operation

After policy propagation, read should return Permission Denied.

Key rule

Google Cloud evaluates applicable Deny policies before Allow policies. An applicable Deny can block a permission even when a role grants it.

The scope must be recorded correctly: organization-level inheritance is not proven by a project-level Deny or a simulator-only result.

Official: IAM Deny policies overview and Deny access guide

SCENARIO 04

GCP-INC-1004 | Prove Allow first, then Deny

If the before-test fails, stop and troubleshoot the grant/impersonation. A failing first read is not proof of Deny precedence.

A BEFORE DENY: READ SHOULD SUCCEED

CONFIRM OBJECT VIEWER ALLOW

```
SA_EMAIL="$(terraform output -raw service_account_email)"
BUCKET="$(terraform output -raw bucket_name)"
OBJ="gs://${BUCKET}/internal-reports/report.txt"

gcloud --impersonate-service-account="$SA_EMAIL" \
  storage cat "$OBJ"
```

Expected: synthetic report text is printed.

B ENABLE DENY + REPEAT THE SAME READ

ATTACH APPROVED DENY POLICY

```
# In terraform.tfvars choose an approved scope
enable_iam_deny_policy = true
deny_policy_scope = "project"
# Organization scope requires numeric org ID + authorization

terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan

# After propagation, rerun the exact same storage cat.
# Expected: Permission Denied
```

Scope label is mandatory

Record whether evidence is project-level, organization-level, or simulator-only.

Use Policy Troubleshooter

Select principal + object + storage.objects.get. Terraform Deny policy uses canonical v2 ID storage.googleapis.com/objects.get.

After evidence: set enable_iam_deny_policy = false and apply, or destroy the lab. Verify the policy is removed.

Supplied lab guide pages 11–12; official IAM Deny and Policy Troubleshooter docs

SUPPORT METHOD

Policy Troubleshooter: explain the access decision

Use the same principal, exact resource and permission as the failing operation.

PRINCIPAL

Night Ops service account

→

RESOURCE

The exact bucket object

→

PERMISSION

storage.objects.get

Inspect both policy paths

- Allow: which binding grants the permission?
- Deny: which applicable rule blocks it?
- Scope: project, folder or organization?

Interpret results carefully

- Permissions may be unknown if policy visibility is missing.
- Policy Troubleshooter does not evaluate Cloud Storage ACLs.
- VPC Service Controls issues need separate analysis.

Do not infer a root cause from “Permission denied” alone: verify identity, resource, permission, Allow, Deny and policy scope.

Official: Policy Troubleshooter troubleshooting IAM permissions

SUPPORT METHOD

How to read command outputs during support

Commands should answer a specific support question, not just generate more text.

Command / view	Question answered	Record in evidence
gcloud config get-value project	Which project is my CLI using?	Project ID / timestamp
terraform output -raw vm_zone	Where did Terraform create the VM?	Zone + VM name
compute instances list / describe	Does the VM exist, and what is its state?	Name, status, zone, network interfaces
sql instances describe	What managed DB was provisioned?	Name, region, engine, tier / config
get-iam-policy / instance IAM policy	Which role is granted and at what scope?	Principal, role, resource scope
storage cat before / after Deny	Did the same object read change?	Success output / exact error + scope

Evidence rule: save actual output. Do not copy illustrative IDs, bucket names or expected errors as if they came from your environment.

Supplied GCP Day 5 Student Lab Reference; official gcloud / IAM documentation

Evidence checklist: what a good support record contains

A student should be able to explain the issue and prove the result without asking the trainer to reproduce it.

INC-1001 | Service boundary

- Cloud SQL metadata
- VM hostname + OS release
- Conclusion: who operates the host OS?

INC-1002 | Discovery

- Wrong context result
- Correct project + zone result
- VM details including IP posture

INC-1003 | Least privilege

- IAM before-state showing Editor
- After-state showing scope change
- Allowed describe + denied list result

INC-1004 | Deny precedence

- Successful read before Deny
- Policy scope and Viewer Allow
- Denied read + Troubleshooter evidence

For every incident: actual command/output + Console screenshot + root cause + remediation + result + cleanup status.

SAFETY

Cost and security guardrails

The lab demonstrates production-relevant controls; run it only with the guardrails below.

Project & billing

- Use the allocated training project ID.
- Confirm billing and API permissions.
- VM, disk, Cloud SQL and optional resources can incur charges.

Identity safety

- Dedicated lab service account only.
- Never create/download a service-account JSON key.
- Do not use a production principal.

Terraform hygiene

- Inspect plan before every apply.
- Keep tfvars and state files private.
- Review destroy plan before approving cleanup.

Organization-level Deny is high-impact

Only use an approved training organization and the required policy-admin permissions. Otherwise, use a project-level fallback if allowed and label the evidence accurately.

Optional features default off: Cloud SQL, IAM Deny and second-project VM. Keep them off until their scenario is ready.

Supplied student lab guide pages 3, 6, 11 and 13; official Terraform and IAM docs

Cleanup sequence and common blockers

Do not leave training resources running after evidence is collected.

SAFE CLEANUP

REVIEW BEFORE DESTROYING

```
terraform state list
terraform plan -destroy
terraform destroy

# Confirm expected resources are gone.
# Remove any manual resources separately.
```

Destroy only lab-owned resources

Inspect the destroy plan and stop if unrelated resources are listed.
Ensure organization-scope policy is removed.

FAST TROUBLESHOOTING

API / permission error	Enable required API or obtain correct permission.
Terraform not found	Install via official instructions; check version.
IAP SSH fails	Check user grant, OS Login, firewall and zone.
Impersonation fails	Check signed-in email and Token Creator grant.
Deny not effective yet	Wait for propagation; verify same SA and scope.

Completion = evidence saved + optional resources disabled/removed + lab environment cleaned + result summarized.

Supplied lab guide page 13; official Terraform command reference

REFERENCE

Quick command crib sheet

Use these as reusable patterns. Replace variables only with values confirmed in your project and Terraform outputs.

CONTEXT + TERRAFORM

```
# Context
gcloud auth list
gcloud config get-value project
gcloud config set project YOUR_EXISTING_PROJECT_ID

# Terraform lifecycle
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
```

COMPUTE + CLOUD SQL

```
# Resource discovery
gcloud compute instances list --project="$PROJECT_ID" \
  --zones="$VM_ZONE" --format="table(name,status,zone)"

gcloud compute instances describe "$VM_NAME" \
  --project="$PROJECT_ID" --zone="$VM_ZONE"

# SQL metadata
gcloud sql instances describe "$SQL_NAME" --project="$PROJECT_ID"
```

IAM + STORAGE + CLEANUP

```
# IAM before / after
gcloud projects get-iam-policy "$PROJECT_ID" \
  --flatten="bindings[].members" \
  --filter="bindings.members:serviceAccount:${SA_EMAIL}"

# Object read under impersonation
gcloud --impersonate-service-account="$SA_EMAIL" \
  storage cat "gs://${BUCKET}/internal-reports/report.txt"

# Cleanup after checking plan
terraform plan -destroy
terraform destroy
```

For exact sequence, variable names and Console steps, follow the supplied student lab reference guide.

Official gcloud references are linked on scenario slides; Terraform guide

REVIEW

Knowledge check: can you explain the why?

Try to answer each question before opening the answer key on the next slide.

1. Why can a VM exist even when a query in one zone returns no matching resource?

2. Which cloud service lets the customer inspect the guest OS in this lab?

3. What is the difference between a principal, a role and a resource in IAM?

4. Why should INC-1004 prove a successful object read before applying Deny?

5. What proves true organization-level Deny inheritance?

6. Why is reviewing terraform plan -destroy important?

Strong support answers include the command, observed result, interpretation, scope and next safe action.

REVIEW

Knowledge check: answer key

Use these model answers to check understanding, not to substitute for evidence from your own lab.

1. The project or zone filter may be wrong. Confirm identity, project, actual zone and filters before creating another VM.

2. Compute Engine. It gives guest-OS access when SSH / OS Login / IAP and permissions are configured.

3. Principal = actor; role = collection of permissions; resource = object those permissions apply to.

4. A valid Allow baseline is necessary. If the read already fails, the error cannot prove Deny precedence.

5. An approved Deny policy attached at organization scope and inherited through the hierarchy; a project policy does not prove this.

6. It prevents accidental deletion of unrelated infrastructure and confirms only lab-owned resources are targeted.

Official: Google Cloud overview, IAM and Deny docs; supplied lab guide

RESOURCES

Further study: official docs + KodeKloud learning videos

Use official documentation for current syntax and feature behavior; use guided videos to reinforce concepts.

OFFICIAL GOOGLE CLOUD REFERENCES

- Google Cloud overview / hierarchy
- Compute Engine instances
- Cloud SQL for PostgreSQL
- Cloud Storage overview
- VPC overview
- IAM roles + Deny
- Policy Troubleshooter
- Terraform on Google Cloud
- AWS account model + SCPs
- Azure resource hierarchy
- Azure deny assignments (scope + limits)

KODEKLOUD: GCP LEARNING PATH

Recommended topics: resource hierarchy, compute, databases, object storage, APIs, security and GCP architecture. Look for the GCP Digital Cloud Leader / GCP Core Fundamentals video modules.

Open the learning path

KODEKLOUD: CLOUD LEARNING PATH

For additional AWS / Azure comparison, revisit cloud computing fundamentals, cloud economics, shared responsibility, storage, networking, IAM and compute services.

Open the cloud learning path

Reference links verified for this deck on 23 October 2026. Product availability, permissions, service tiers and CLI syntax can change; use the live docs when implementing.